U07 · La API Stream

Ya sabes recorrer colecciones con bucles e iteradores. La API Stream (Java 8) te ofrece otra forma de procesarlas: en vez de describir *cómo* recorrerlas paso a paso, describes *qué* quieres obtener — el mismo cambio de enfoque que ya viste con SQL.

Todos los ejemplos de esta unidad usan el mismo escenario: una lista `menu` con platos de un catering.

```
public class Plato {
    public enum Tipo { CARNE, PESCADO, OTRO }
    private final String nombre;
    private final boolean vegano;
    private final int calorias;
    private final Tipo tipo;
    // constructor y getters...
}

List<Plato> menu = List.of(
    new Plato("cerdo", false, 800, Plato.Tipo.CARNE),
    new Plato("cordero", false, 700, Plato.Tipo.CARNE),
    new Plato("pollo", false, 400, Plato.Tipo.CARNE),
    new Plato("arroz", true, 350, Plato.Tipo.OTRO),
    new Plato("trucha", false, 300, Plato.Tipo.PESCADO),
    new Plato("salmón", false, 450, Plato.Tipo.PESCADO)
);
```

1. El mismo problema, dos estilos

Imagina una lista de platos de un menú, cada uno con nombre, calorías y tipo, y quieres los nombres de los platos con menos de 400 calorías, ordenados por calorías.

Con colecciones (imperativo — describes los pasos):

```
List<Plato> ligeros = new ArrayList<>();
for (Plato p : menu) {
    if (p.getCalorias() < 400) ligeros.add(p);
}
Collections.sort(ligeros, Comparator.comparing(Plato::getCalorias));
List<String> nombres = new ArrayList<>();
for (Plato p : ligeros) nombres.add(p.getNombre());
```

Con Streams (declarativo — describes el resultado):

```
List<String> nombres = menu.stream()
    .filter(p -> p.getCalorias() < 400)
    .sorted(Comparator.comparing(Plato::getCalorias))
    .map(Plato::getNombre)
    .collect(Collectors.toList());
```

Menos código, más legible, y si algún día necesitas paralelizarlo, el cambio es trivial: `menu.stream()` → `menu.parallelStream()`.

2. Crear un stream

La forma más habitual es a partir de una colección ya existente:

```
Stream<Plato> s = menu.stream();
```

Pero también puedes crear streams de otras formas:

```
Stream<String> deValores = Stream.of("Java", "Lambdas", "Streams");
Stream<String> vacio = Stream.empty();

int[] numeros = {1, 2, 3, 4, 5};
int suma = Arrays.stream(numeros).sum();          // stream desde un array

Stream<String> lineas = Files.lines(Paths.get("datos.txt"));   // stream desde un
fichero
```

Y streams **infinitos**, generados sobre la marcha (por eso casi siempre van acompañados de `limit(n)`):

```
Stream.iterate(0, n -> n + 2)        // 0, 2, 4, 6, 8...
    .limit(10)
    .forEach(System.out::println);

Stream.generate(Math::random)        // números aleatorios, sin relación entre sí
    .limit(5)
    .forEach(System.out::println);
```

3. El pipeline: intermedias + una final

Trabajar con un stream sigue siempre la misma estructura de 3 pasos: **origen** → **operaciones intermedias** (encadenables, tantas como quieras) → **una operación final** que dispara la ejecución.

```
List<String> top3 = menu.stream()          // origen
    .filter(p -> p.getCalorias() > 300)     // intermedia
    .map(Plato::getNombre)                   // intermedia
    .limit(3)                                 // intermedia
    .collect(Collectors.toList());            // FINAL: aquí se ejecuta todo
```

> **Los streams son "perezosos"**
>
> Nada se ejecuta hasta que llega la operación final. Un stream con solo operaciones intermedias no hace absolutamente nada por sí solo — es solo una "receta" pendiente de cocinar. Esto le permite a Java optimizar el pipeline entero antes de procesar un solo elemento.

4. Filtrar: `filter`, `distinct`, `limit`, `skip`

```
menu.stream().filter(p -> p.esVegano());        // solo los que cumplen la
condición
menu.stream().distinct();                        // elimina duplicados (usa equals())
menu.stream().limit(5);                          // como mucho 5 elementos
menu.stream().skip(2);                           // se salta los 2 primeros
```

5. Transformar: `map` y `flatMap`

`map` transforma cada elemento en otro (posiblemente de otro tipo):

```
List<String> nombres = menu.stream()
    .map(Plato::getNombre)
    .collect(Collectors.toList());
```

`flatMap` es para cuando cada elemento produce a su vez **varios** elementos (por ejemplo, una lista de listas, o una lista de frases que quieres convertir en una lista de palabras) — "aplana" el resultado en un único stream:

```
List<String> palabras = frases.stream()
    .flatMap(frase -> Arrays.stream(frase.split(" ")))
    .distinct()
    .collect(Collectors.toList());
```

6. Comprobar: `anyMatch`, `allMatch`, `noneMatch`, `findFirst`

```
boolean hayVeganos = menu.stream().anyMatch(Plato::esVegano);       // ¿al menos
uno cumple?
boolean todosBajos = menu.stream().allMatch(p -> p.getCalorias() < 1000);  // ¿todos
cumplen?
boolean ningunoCaro = menu.stream().noneMatch(p -> p.getCalorias() > 2000); //
¿ninguno cumple?

Optional<Plato> primero = menu.stream().filter(Plato::esVegano).findFirst();
```

7. Reducir: `reduce`, `count`, `sum`

`reduce` combina todos los elementos en un único resultado, aplicando la misma operación acumulativa una y otra vez:

```
int totalCalorias = menu.stream()
    .map(Plato::getCalorias)
    .reduce(0, (acumulado, cal) -> acumulado + cal);   // arranca en 0, va sumando
```

Equivale, paso a paso, a:

```
int resultado = 0;
for (Plato p : menu) {
    resultado = resultado + p.getCalorias();
}
```

Para las operaciones más comunes, no hace falta escribir el `reduce` a mano — ya existen atajos:

```
long numPlatos = menu.stream().count();
int total = menu.stream().mapToInt(Plato::getCalorias).sum();
OptionalInt max = menu.stream().mapToInt(Plato::getCalorias).max();
```

8. Recolectar: `collect` y `Collectors`

`collect` es la operación final más flexible: convierte el stream en una colección, un `String`, un `Map`… según el `Collector` que le pases.

```
List<String> lista = stream.collect(Collectors.toList());
Set<String> conjunto = stream.collect(Collectors.toSet());
String texto = nombres.stream().collect(Collectors.joining(", "));   // "cerdo,
cordero, pollo"

// Estadísticas directas
long cuenta = menu.stream().collect(Collectors.counting());
double media = menu.stream().collect(Collectors.averagingInt(Plato::getCalorias));
```

Agrupar con `groupingBy`

El equivalente al `GROUP BY` de SQL:

```
Map<Plato.Tipo, List<Plato>> porTipo = menu.stream()
    .collect(Collectors.groupingBy(Plato::getTipo));
// {CARNE=[cerdo, cordero, pollo], PESCADO=[trucha, salmón], OTRO=[arroz, pizza...]}
```

La función de clasificación puede ser tan compleja como necesites, con una lambda:

```
Map<String, List<Plato>> porNivelCalorico = menu.stream()
    .collect(Collectors.groupingBy(p -> {
        if (p.getCalorias() <= 400) return "DIETA";
        if (p.getCalorias() <= 700) return "NORMAL";
        return "PESADO";
    }));
```

Y `groupingBy` admite un segundo nivel, para agrupar dentro de cada grupo:

```
Map<Plato.Tipo, Map<String, List<Plato>>> porTipoYNivel = menu.stream()
    .collect(Collectors.groupingBy(Plato::getTipo,
             Collectors.groupingBy(p -> p.getCalorias() <= 400 ? "DIETA" :
"NORMAL")));
```

9. `Optional<T>`: evitar el `null`

Los métodos que “podrían no encontrar nada” (`findFirst`, `findAny`, `max`…) no devuelven `null` — devuelven un `Optional<T>`, un contenedor que puede tener o no un valor, obligándote a pensar explícitamente en el caso “no hay resultado”:

```java
Optional<Plato> masCalorico = menu.stream()
    .max(Comparator.comparing(Plato::getCalorias));

if (masCalorico.isPresent()) {
    System.out.println(masCalorico.get());
}
// o, más idiomático:
masCalorico.ifPresent(p -> System.out.println(p.getNombre()));
String nombre = masCalorico.map(Plato::getNombre).orElse("Sin platos");
```

10. Streams en paralelo

Cambiar `.stream()` por `.parallelStream()` (o llamar a `.parallel()` sobre un stream ya creado) reparte el procesamiento entre varios núcleos de la CPU automáticamente:

```java
long total = menu.parallelStream()
    .mapToInt(Plato::getCalorias)
    .sum();
```

> ⚠ **El paralelismo no siempre compensa**
>
> Paralelizar tiene su propio coste (dividir el trabajo, coordinar los resultados). Para colecciones pequeñas, un stream paralelo puede ser incluso **más lento** que uno secuencial. Además, la operación que apliques debe ser independiente entre elementos (sin depender del orden ni modificar variables compartidas) — igual que cualquier código concurrente, es fácil introducir errores sutiles si no se tiene cuidado.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA6. Escribe programas que manipulen información con tipos avanzados de datos…	a) programas con arrays · b) librerías de tipos avanzados · c) listas · d) iteradores · e) características y ventajas de las colecciones · f) clases y métodos genéricos

Boletín inicial | Boletín intermedio | Extras